Αναλυτική επεξήγηση του κώδικα

smart_meter.ino — ESP32-C3 Mini · ZMPT101B (GPIO2) · SCT-013 (GPIO0) · MQTT. Οι αριθμοί γραμμών αντιστοιχούν ακριβώς στο αρχείο. Όλα τα μετρημένα νούμερα προέρχονται από τη συγκεκριμένη διάταξη.

1. Η γενική ιδέα

Και οι δύο αισθητήρες βγάζουν ένα **ημιτονοειδές σήμα τάσης** που ταλαντώνεται γύρω από ένα σταθερό επίπεδο ηρεμίας (~1.65 V, το μισό των 3.3 V). Ο ADC διαβάζει μόνο θετικές τάσεις, γι' αυτό χρειάζεται αυτό το «ανέβασμα».

Ο κώδικας παίρνει **~2800 δείγματα ανά κανάλι** μέσα σε 39 ημιπεριόδους του δικτύου, αφαιρεί από κάθε δείγμα το επίπεδο ηρεμίας, και βγάζει στατιστικά: μέση τετραγωνική τιμή για τα rms, μέσο όρο του γινομένου τάσης×ρεύματος για την ισχύ.

```
setup()
  +-- logBegin()  -> USB CDC + UART0, 115200
  +-- ADC 12-bit, εξασθένηση 11dB και στα δύο pins
  +-- netSetup()  -> WiFi, device id από MAC, ρυθμίσεις broker
  +-- measure()   <- "τυφλή" μέτρηση: κλειδώνει τα φίλτρα offset
  |
loop()  (κάθε ~0.6 s)
  +-- netLoop()  -> επανασύνδεση WiFi/MQTT αν χρειάζεται, mqtt.loop()
  |
  +-- measure() ------------------------------------------+
  |     1. warm-up 60 ms  -> ανανέωση offsetV/offsetI      |
  |     2. δειγματοληψία ως 40 μηδενισμούς τάσης (~2800 δείγμ.)|
  |          sV,sI -> αφαίρεση offset -> fV,fI             |
  |          ring buffer -> μετατόπιση φάσης PHASE_SHIFT   |
  |          sumV += fV^2 , sumI += fI^2 , sumP += vS*iS   |
  |     3. sqrt/N, x VCAL, x ICAL  -> V, I, P, S, PF       |
  +--------------------------------------------------------+
  |
  +-- energy_Wh += P * dt      (dt από millis())
  +-- logf(...)  -> σειριακή
  +-- αν V==0: currentOnlyReport()   | αν I>1A: phaseSweep()
  +-- κάθε 5 s: publishState() -> MQTT
  +-- delay(200)
```

2. Η θεωρία σε μισή σελίδα

Ενεργός τιμή (RMS). Το μέσο ενός ημιτόνου είναι μηδέν. Αυτό που μετράμε είναι η τετραγωνική ρίζα του μέσου των τετραγώνων:

$$Vrms = \sqrt{(v1^2 + v2^2 + \dots + vN^2) / N}$$

Πραγματική ισχύς. Δεν είναι Vrms × Irms, αλλά ο μέσος όρος της *στιγμιαίας* ισχύος — τάση επί ρεύμα **στο ίδιο χρονικό σημείο**:

$$P = (v1 \cdot i1 + v2 \cdot i2 + \dots + vN \cdot iN) / N \quad [Watt]$$

Αν το ρεύμα είναι εκτός φάσης με την τάση, τα γινόμενα βγαίνουν κατά διαστήματα αρνητικά και ο μέσος όρος πέφτει — όπως συμβαίνει στην πραγματικότητα. **Γι' αυτό η διόρθωση φάσης του §4 είναι τόσο κρίσιμη:** σφάλμα 28° έδινε 12% λάθος στην ισχύ.

$$S = Vrms \cdot Irms \quad [VA] \qquad PF = P / S$$

Ενέργεια. `energy_Wh += P × dt`, με το dt σε ώρες.

3. Το μετρητικό κομμάτι

3.1 Ακροδέκτες (γραμμές 38–40)

```
// ------------------------------------------------------------------ ΑΚΡΟΔΕΚΤΕΣ
static const int PIN_VOLT = 2;   // ZMPT101B  OUT  (ADC1_CH2)
static const int PIN_CURR = 0;   // SCT-013   sig  (ADC1_CH0)
```

Στον ESP32-C3 μόνο τα **GPIO0–GPIO4** ανήκουν στον **ADC1**. Ο ADC2 μπλοκάρει όταν είναι ενεργό το WiFi — και εδώ το WiFi είναι μονίμως ανοιχτό για το MQTT, οπότε η επιλογή του ADC1 είναι υποχρεωτική.

3.2 Οι σταθερές βαθμονόμησης (γραμμές 42–61)

```
// ------------------------------------------------------------------ ΒΑΘΜΟΝΟΜΗΣΗ
// VCAL: πόσα Volt AC του δικτύου αντιστοιχούν σε 1 V στο pin.
//       Ξεκίνα με 260.0 και διόρθωσε: VCAL_new = VCAL_old * (Υπολύμετρου / Νσειριακής)
float VCAL = 341.0f;

// ICAL: πόσα Ampere αντιστοιχούν σε 1 V στο pin.
//       SCT-013-030 (30A/1V) απευθείας στο pin  -> 30.0
//       SCT-013-010 (10A/1V)                    -> 10.0
//       Αν έβαλες επιπλέον διαιρέτη ΠΑΝΩ στο σήμα (π.χ. /2) -> διπλασίασε το.
float ICAL = 25.75f;

// PHASECAL: διόρθωση διαφοράς φάσης V/I (1.0 = καμία διόρθωση).
//           Ρύθμισέ το με καθαρά ωμικό φορτίο ώστε cos(φ) ~ 1.00 (τυπικά 0.8 – 2.2)
float PHASECAL = 1.0f;

// PHASE_SHIFT: χονδρική διόρθωση φάσης σε ΟΛΟΚΛΗΡΑ δείγματα (το PHASECAL
// καλύπτει μόνο ±1 δείγμα = ±2.5 μοίρες, πολύ λίγο για τη μετατόπιση που
// εισάγει ο μετασχηματιστής του ZMPT101B). Θετικό = καθυστερεί την τάση.
// Βρες την τιμή με τη σάρωση [sweep] που τρέχει αυτόματα υπό φορτίο.
int PHASE_SHIFT = 12;
```

Ο ADC μετράει **Volt στο pin**· τα VCAL/ICAL είναι ο λόγος μετατροπής προς τον πραγματικό κόσμο:

`Vδικτύου = VCAL × Vpin` `Iφορτίου = ICAL × Vpin`

Το ICAL βγήκε 25.75, όχι 30. Ένα SCT-013-030 θεωρητικά δίνει 1 V στα 30 A, αλλά το συγκεκριμένο clamp μετρήθηκε στα ~25.8 A/V. Τέσσερις συγκρίσεις με αμπεροτσιμπίδα σε δύο επίπεδα φορτίου (4.6 A και 8.8 A) έδωσαν 25.99 / 25.77 / 25.86 / 25.41. Η διασπορά του 1% είναι το όριο επαναληψιμότητας της ίδιας της σύγκρισης, όχι σφάλμα του μετρητή — δύο μετρήσεις στο *ίδιο* φορτίο διέφεραν κατά 1.4% μεταξύ τους.

Δύο σημεία, όχι ένα. Με ένα μόνο σημείο δεν ξέρεις αν ο μετρητής είναι γραμμικός ή απλώς κουρδισμένος εκεί. Με 4.6 και 8.8 A — διπλάσιο ρεύμα — και συμφωνία κάτω από 1%, ξέρεις.

3.3 Παράμετροι δειγματοληψίας (γραμμές 63–74)

```
// ------------------------------------------------------------------ ΔΕΙΓΜΑΤΟΛΗΨΙΑ
static const int      HALF_CYCLES  = 40;     // 39 ημιπερίοδοι = 19.5 περίοδοι @50Hz (~390 ms)
static const uint32_t TIMEOUT_MS   = 1500;   // ασφάλεια αν δεν υπάρχει σήμα
static const double   FILT_TAU     = 1024.0; // σταθερά φίλτρου DC offset
static const float    NOISE_V      = 5.0f;   // κάτω από αυτό -> 0 (Volt)
static const float    NOISE_I      = 0.50f;  // κάτω από αυτό -> 0 (μετρημένος θόρυβος ~0.3 A)

// Αν το clamp είναι ανάποδα και δεν βολεύει να το γυρίσεις (π.χ. μόνιμη
// εγκατάσταση), βάλε 1 και αντιστρέφεται το πρόσημο της ισχύος από λογισμικό.
#define INVERT_CURRENT 0

#define PRINT_DEBUG 1                        // 1 = τύπωσε offsets/δείγματα
```

Σταθερά	Τι κάνει
HALF_CYCLES	40 μηδενισμοί = 39 ημιπερίοδοι ≈ 390 ms στα 50 Hz. **Δουλεύει αυτόματα και στα 60 Hz** — τελειώνει απλώς νωρίτερα.
TIMEOUT_MS	Αν δεν υπάρχει σήμα τάσης ο βρόχος δεν κολλάει· επιστρέφει με `ok = false`.
FILT_TAU	Το «βάρος» του φίλτρου που βρίσκει το επίπεδο ηρεμίας (§3.5).
NOISE_V / NOISE_I	Κατώφλι θορύβου. Το `NOISE_I` ανέβηκε από 0.06 σε **0.50**: ο μετρημένος θόρυβος της διάταξης είναι ~0.3 A, και με το αρχικό κατώφλι ο μετρητής μάζευε kWh χωρίς κανένα φορτίο.
INVERT_CURRENT	Αντιστρέφει το πρόσημο της ισχύος από λογισμικό, αν το clamp είναι ανάποδα και δεν βολεύει να γυριστεί.
PRINT_DEBUG	Ανοίγει τα τρία διαγνωστικά του §5.

3.4 Καθολικές μεταβλητές και το struct (γραμμές 76–92)

```
76 // ------------------------------------------------------------------ ΜΕΤΑΒΛΗΤΕΣ
77 static double offsetV = 1650.0;   // DC offset σε mV (ξεκινά στο μισό των 3.3V)
78 static double offsetI = 1650.0;
79
80 static double energy_Wh = 0.0;    // συσσωρευμένη ενέργεια
81 static uint32_t lastEnergy_ms = 0;
82
83 struct Reading {
84   float  vrms;          // V
85   float  irms;          // A
86   float  realPower;     // W
87   float  apparentPower; // VA
88   float  powerFactor;   // 0..1
89   int    samples;
90   int    vmin, vmax;    // ακραίες τιμές του καναλιού τάσης, σε mV (έλεγχος clipping)
91   bool   ok;
92 };
```

Τα `offsetV/offsetI` είναι **static** επίτηδες: κρατούν την τιμή τους ανάμεσα σε κλήσεις, οπότε το φίλτρο δεν ξεκινά από το μηδέν κάθε φορά.

3.5 measure(), βήμα 1: warm-up και το φίλτρο DC offset (γραμμές 136–145)

```
136 static Reading measure() {
137   Reading r = {0, 0, 0, 0, 0, 0, 5000, 0, false};
138
139   const uint32_t tStart = millis();
140
141   // 1) Warm-up: αφήνουμε το φίλτρο DC offset να "κλειδώσει"
142   while (millis() - tStart < 60) {
143     offsetV += (analogReadMilliVolts(PIN_VOLT) - offsetV) / FILT_TAU;
144     offsetI += (analogReadMilliVolts(PIN_CURR) - offsetI) / FILT_TAU;
145   }
```

```
offset += (δείγμα − offset) / 1024
```

Ένα **βαθυπερατό φίλτρο πρώτης τάξης**. Το ημίτονο των 50 Hz σβήνει στον μέσο όρο, ενώ αργές μεταβολές (θερμοκρασία, τροφοδοσία) περνούν και διορθώνονται.

Ο μετρημένος ρυθμός δειγματοληψίας είναι **~7.2 kSPS ανά κανάλι** μέσα στη `measure()` — δύο αναγνώσεις ADC ανά επανάληψη. (Η μονοκάναλη `currentOnlyReport()` φτάνει τα 14.8 kSPS.) Άρα η συχνότητα αποκοπής είναι `7200 / (2π × 1024) ≈ 1.1` Hz, περίπου 45 φορές κάτω από το δίκτυο, με ~145 δείγματα ανά περίοδο.

Το offset είναι το καλύτερο διαγνωστικό της συνδεσμολογίας. Πρέπει να είναι ~1650 mV και στα δύο κανάλια. Στη διάρκεια του στησίματος, ένα `offsetV` στα 2500 mV αποκάλυψε ότι το ZMPT101B τροφοδοτούνταν με 5 V αντί για 3.3 V, και ένα `offsetI` στα 385 mV ότι είχε αποσυνδεθεί η αντίσταση του διαιρέτη.

3.6 measure(), βήμα 2: συγχρονισμός στους μηδενισμούς (γραμμές 147–188)

```
147   // 2) Κύρια δειγματοληψία, συγχρονισμένη σε μηδενισμούς της τάσης
148   const int PBUF = 32;                 // > |PHASE_SHIFT| + 1
149   static double pbufV[32], pbufI[32];
150   int tick = 0;
151
152   double sumV = 0, sumI = 0, sumP = 0;
153   double lastFilteredV = 0;
154   int    crossCount = 0;
155   int    n = 0;
156   bool   started = false;
157
158   // Αρχικοποίηση προσήμου από το τρέχον δείγμα, ώστε ο πρώτος μηδενισμός
159   // που θα ανιχνευτεί να είναι πραγματικός -> αθροίζουμε ακέραιο αριθμό
160   // ημιπεριόδων και το RMS βγαίνει ακριβές.
161   bool   lastSign = (analogReadMilliVolts(PIN_VOLT) - offsetV) > 0;
162
163   while (crossCount < HALF_CYCLES) {
164     if (millis() - tStart > TIMEOUT_MS) return r;   // ok = false
165
166     int    rawV = analogReadMilliVolts(PIN_VOLT);
167     double sV = rawV;
168     double sI = analogReadMilliVolts(PIN_CURR);
169
170     if (rawV < r.vmin) r.vmin = rawV;
171     if (rawV > r.vmax) r.vmax = rawV;
172
173     offsetV += (sV - offsetV) / FILT_TAU;
174     offsetI += (sI - offsetI) / FILT_TAU;
175
176     double fV = sV - offsetV;      // AC συνιστώσα τάσης  (mV στο pin)
177     double fI = sI - offsetI;      // AC συνιστώσα ρεύματος (mV στο pin)
178
179     bool sign = (fV > 0);
180     if (sign != lastSign) {
181       crossCount++;
182       if (!started) {              // ξεκίνα να αθροίζεις από τον 1ο μηδενισμό
183         started = true;
184         sumV = sumI = sumP = 0;
185         n = 0;
186       }
187     }
188     lastSign = sign;
```

Γιατί μηδενισμοί; Το RMS βγαίνει σωστό μόνο πάνω σε **ακέραιο αριθμό ημιπεριόδων**. Γι' αυτό ο κώδικας δεν μετράει χρόνο αλλά εναλλαγές προσήμου της τάσης.

Η γραμμή 161 διαβάζει ένα δείγμα πριν τον βρόχο, ώστε το `lastSign` να ξεκινά με το πραγματικό πρόσημο· αλλιώς θα μετριόταν ψεύτικος μηδενισμός στη μέση ενός κύματος. Στη γραμμή 182 μηδενίζονται τα αθροίσματα με τον πρώτο πραγματικό μηδενισμό.

Η γραμμή 166 διαβάζει με `analogReadMilliVolts()` και όχι `analogRead()`: εφαρμόζει τη **βαθμονόμηση από τα eFuse του τσιπ** και επιστρέφει millivolt. Η ακρίβειά της επιβεβαιώθηκε στην πράξη — διάβαζε 1652 mV στον διαιρέτη 10k/10k, όπου η θεωρητική τιμή είναι 1650.

Εδώ μετριέται και το `vmin/vmax` του καναλιού τάσης, που τροφοδοτεί τον έλεγχο κορεσμού.

3.7 measure(), βήμα 3: ring buffer, φάση και αθροίσματα (γραμμές 190–209)

```
    // Γράψε τα δείγματα στον κυκλικό buffer και διάβασε τα μετατοπισμένα.
    int pi = tick % PBUF;
    pbufV[pi] = fV;
    pbufI[pi] = fI;
    int kv = (PHASE_SHIFT > 0) ?  PHASE_SHIFT : 0;   // καθυστέρηση τάσης
    int ki = (PHASE_SHIFT < 0) ? -PHASE_SHIFT : 0;   // καθυστέρηση ρεύματος
    double v0 = pbufV[(pi - kv     + PBUF) % PBUF];
    double v1 = pbufV[(pi - kv - 1 + PBUF) % PBUF];
    double phaseShiftedV = v1 + PHASECAL * (v0 - v1);   // λεπτή ρύθμιση
    double iShifted      = pbufI[(pi - ki + PBUF) % PBUF];

    if (started && tick >= PBUF) {
      sumV += fV * fV;
      sumI += fI * fI;
      sumP += phaseShiftedV * iShifted;
      n++;
    }
    tick++;
    lastFilteredV = fV;
  }
```

Ο κυκλικός buffer 32 θέσεων κρατά τα τελευταία δείγματα ώστε να μπορεί να χρησιμοποιηθεί η τάση **καθυστερημένη κατά PHASE_SHIFT δείγματα**. Θετική τιμή καθυστερεί την τάση, αρνητική το ρεύμα. Το PHASECAL κάνει γραμμική παρεμβολή *μέσα* σε ένα δείγμα, για λεπτομέρεια κάτω από 2.5°.

Η συνθήκη `tick >= PBUF` εξασφαλίζει ότι δεν αθροίζονται δείγματα πριν γεμίσει ο buffer.

3.8 measure(), βήμα 4: από mV σε Volt, Ampere και Watt (γραμμές 211–236)

```
  if (n < 10) return r;

  // 3) Υπολογισμοί (mV -> V, μετά επί τους συντελεστές βαθμονόμησης)
  double vrms_pin = sqrt(sumV / n) / 1000.0;   // V στο pin
  double irms_pin = sqrt(sumI / n) / 1000.0;   // V στο pin

  r.vrms      = VCAL * vrms_pin;
  r.irms      = ICAL * irms_pin;
  r.realPower = VCAL * ICAL * (sumP / n) / 1.0e6;

  if (r.vrms < NOISE_V) { r.vrms = 0; r.realPower = 0; }
  if (r.irms < NOISE_I) { r.irms = 0; r.realPower = 0; }

#if INVERT_CURRENT
  r.realPower = -r.realPower;
#endif

  r.apparentPower = r.vrms * r.irms;
  r.powerFactor   = (r.apparentPower > 0.5f) ? (r.realPower / r.apparentPower) : 0.0f;
  if (r.powerFactor >  1.0f) r.powerFactor =  1.0f;
  if (r.powerFactor < -1.0f) r.powerFactor = -1.0f;

  r.samples = n;
  r.ok = true;
  return r;
}
```

- `sqrt(sumV/n)` → rms σε **mV στο pin**
- `/1000` → **V στο pin** · `× VCAL` → **Volt δικτύου**

Για την ισχύ, το `sumP/n` είναι σε mV², άρα ÷10□ → V^2_{pin}, και `× VCAL × ICAL` → **Watt**.

Οι γραμμές 221–222 είναι η πύλη θορύβου: κάτω από το κατώφλι μηδενίζεται και η ισχύς, αλλιώς αθροίζεται ενέργεια από τον θόρυβο. Ο PF υπολογίζεται μόνο με ουσιαστικό φορτίο (S > 0.5 VA) και περιορίζεται στο ±1.

4. Η διόρθωση φάσης — το μεγαλύτερο εύρημα

Με καθαρά ωμικό φορτίο (θερμάστρα 2 kW) ο μετρητής έδειχνε **PF = 0.88** αντί για 1.00. Αυτό αντιστοιχεί σε γωνία **27.9°** και σε **12% σφάλμα στην ισχύ** — 1847 W αντί για 2087 W — που θα κουβαλιόταν σε κάθε μέτρηση kWh για πάντα.

Η αιτία δεν είναι η δειγματοληψία. Ο ADC διαβάζει τάση και ρεύμα με ~70 μs διαφορά, που στα 50 Hz είναι μόλις 1.3°. Η μετατόπιση προέρχεται από τον μετασχηματιστή και το φίλτρο του ZMPT101B.

Το κλασικό PHASECAL δεν έφτανε. Μετατοπίζει το πολύ ±1 δείγμα, δηλαδή ±2.5° στα 50 Hz — ένα δέκατο της διόρθωσης που χρειαζόταν. Γι' αυτό προστέθηκε το PHASE_SHIFT σε ολόκληρα δείγματα.

4.1 phaseSweep(): αυτόματη εύρεση (γραμμές 400–441)

```
400 #define SWEEP_N      2048
401 #define SWEEP_MAX_K  24
402
403 static void phaseSweep() {
404   static float bufV[SWEEP_N], bufI[SWEEP_N];
405
406   uint32_t t0 = micros();
407   for (int n = 0; n < SWEEP_N; n++) {
408     double sV = analogReadMilliVolts(PIN_VOLT);
409     double sI = analogReadMilliVolts(PIN_CURR);
410     offsetV += (sV - offsetV) / FILT_TAU;
411     offsetI += (sI - offsetI) / FILT_TAU;
412     bufV[n] = sV - offsetV;
413     bufI[n] = sI - offsetI;
414   }
415   float fs = SWEEP_N / ((micros() - t0) / 1.0e6f);     // πραγματικός ρυθμός
416
417   const int a = SWEEP_MAX_K, b = SWEEP_N - SWEEP_MAX_K;
418   double sv = 0, si = 0;
419   for (int n = a; n < b; n++) { sv += (double)bufV[n]*bufV[n]; si += (double)bufI[n]*bufI[n]; }
420   double vr = sqrt(sv / (b - a)), ir = sqrt(si / (b - a));
421
422   if (vr < 50.0 || ir < 20.0) { logf("[sweep] πολύ μικρό σήμα - βάλε φορτίο\n"); return; }
423
424   // ΠΡΟΣΟΧΗ: η σάρωση δουλεύει πάνω σε ωμά δείγματα, ΧΩΡΙΣ να εφαρμόζει το
425   // PHASE_SHIFT. Άρα το bestK είναι η απόλυτη τιμή που πρέπει να μπει εκεί,
426   // όχι κάτι που προστίθεται στην τρέχουσα.
427   int bestK = 0; double bestPF = 0, pf0 = 0, pfCur = 0;
428   for (int k = -SWEEP_MAX_K; k <= SWEEP_MAX_K; k++) {
429     double sp = 0;
430     for (int n = a; n < b; n++) sp += (double)bufV[n - k] * bufI[n];
431     double pf = sp / ((b - a) * vr * ir);
432     if (k == 0)           pf0   = pf;
433     if (k == PHASE_SHIFT) pfCur = pf;
434     if (fabs(pf) > fabs(bestPF)) { bestPF = pf; bestK = k; }
435   }
436
437   logf("[sweep] fs=%.0fHz | αδιορθωτο(0)=%.3f | τρεχον(%d)=%.3f | ΒΕΛΤΙΣΤΟ PHASE_SHIFT=%d -> PF=%.3f (%.1f μοιρες)\n",
438        fs, pf0, PHASE_SHIFT, pfCur, bestK, bestPF,
439        bestK * 360.0f * 50.0f / fs);
440   if (bestPF < 0) logf("[sweep] ΠΡΟΣΟΧΗ: αρνητικο PF -> το clamp ειναι αναποδα\n");
441 }
```

Καταγράφει 2048 δείγματα (~290 ms) και δοκιμάζει **49 μετατοπίσεις** (±24 δείγματα) πάνω στην ίδια κυματομορφή. Με ωμικό φορτίο, η μετατόπιση που μεγιστοποιεί το PF είναι εξ ορισμού η σωστή — ξέρουμε ότι η απάντηση οφείλει να είναι 1.00.

```
[sweep] fs=7737Hz | αδιορθωτο(0)=-0.878 | τρεχον(12)=-0.998 | ΒΕΛΤΙΣΤΟ PHASE_SHIFT=12 -> PF=-0.998 (27.9 μοιρες)
[sweep] ΠΡΟΣΟΧΗ: αρνητικο PF -> το clamp ειναι αναποδα
```

Η σάρωση δουλεύει πάνω σε **ωμά δείγματα, χωρίς να εφαρμόζει το PHASE_SHIFT**. Άρα η τιμή που προτείνει είναι απόλυτη — μπαίνει ως έχει, δεν προστίθεται στην τρέχουσα.

Το αποτέλεσμα επαληθεύτηκε σε δύο επίπεδα ισχύος, 1085 W και 2087 W: PF = 1.00 και στα δύο, δηλαδή η διόρθωση δεν είναι «κουρδισμένη» σε ένα σημείο.

5. Τα διαγνωστικά εργαλεία

5.1 Ο έλεγχος κορεσμού: crest factor (γραμμές 506–517)

```
  // Crest factor = κορυφή / ενεργός τιμή. Καθαρό ημίτονο -> 1.41.
  // Πολύ μικρότερο σημαίνει ότι το σήμα "πλακώνει" (saturation του τελεστικού).
  float vrmsPin_mV = (VCAL > 0) ? (r.vrms / VCAL * 1000.0f) : 0.0f;
  float crest = (vrmsPin_mV > 1.0f) ? ((r.vmax - r.vmin) / 2.0f) / vrmsPin_mV : 0.0f;

  logf("[dbg] offsetV=%.0fmV offsetI=%.0fmV samples=%d | V: %d..%d mV ptp=%d CF=%.2f %s\n",
       offsetV, offsetI, r.samples, r.vmin, r.vmax, r.vmax - r.vmin, crest,
       // Το ταβάνι του ADC είναι ~2850 mV: αν το ακουμπάμε, κόβεται η κορυφή
       // ΑΝΕΞΑΡΤΗΤΑ από το τι δείχνει το crest factor.
       (r.vmax > 2800 || r.vmin < 150) ? "<<< ΚΟΡΥΦΗ ΣΤΟ ΤΑΒΑΝΙ - χαμηλωσε κι αλλο"
         : (crest > 1.33f && crest < 1.50f) ? "OK"
         : (crest > 0.0f ? "<<< σχημα ακομα στραβο" : ""));
```

Το **crest factor** είναι ο λόγος κορυφή/ενεργός τιμή. Για καθαρό ημίτονο είναι πάντα **1.41**· όσο πλησιάζει το 1.00, τόσο πιο «πλακωμένο» είναι το κύμα.

Στο στήσιμο αυτής της διάταξης το CF ήταν **1.10** — σχεδόν τετραγωνικό κύμα. Το ποτενσιόμετρο ήταν πολύ ψηλά *και* το σήμα κεντραρισμένο στα 2.5 V, οπότε η κορυφή χτυπούσε στο ταβάνι του ADC.

Προσοχή στο κριτήριο: μόνο το CF δεν αρκεί. Σε μια φάση της ρύθμισης το CF ανέβηκε στο 1.33 ενώ το `max` παρέμενε καρφωμένο στα 2853 mV — ακόμα κοβόταν. Γι' αυτό ο έλεγχος κοιτάζει **πρώτα** αν η κορυφή ακουμπάει το ταβάνι και μόνο μετά το CF. Ένα νούμερο που δεν κουνιέται καθόλου από μέτρηση σε μέτρηση είναι όριο, όχι σήμα.

5.2 currentOnlyReport(): μέτρηση ρεύματος χωρίς τάση (γραμμές 376–394)

```
376 static void currentOnlyReport(uint32_t windowMs) {
377   const uint32_t t0 = millis();
378   double sumI = 0;
379   int n = 0, mn = 5000, mx = 0;
380
381   while (millis() - t0 < windowMs) {
382     int raw = analogReadMilliVolts(PIN_CURR);
383     if (raw < mn) mn = raw;
384     if (raw > mx) mx = raw;
385     offsetI += (raw - offsetI) / FILT_TAU;
386     double fI = raw - offsetI;
387     sumI += fI * fI;
388     n++;
389   }
390
391   float irms = (n > 9) ? (float)(ICAL * sqrt(sumI / n) / 1000.0) : 0.0f;
392   logf("[I-only] Irms=%.3f A | ptp=%dmV (%d..%d) | %d δείγματα σε %ums -> %.1f kSPS\n",
393        irms, mx - mn, mn, mx, n, windowMs, n / (float)windowMs);
394 }
```

Το παράθυρο της `measure()` ορίζεται από τους μηδενισμούς της τάσης. Χωρίς σήμα τάσης, ο θόρυβος δίνει τυχαίες εναλλαγές προσήμου και το παράθυρο κλείνει μετά από ~70 δείγματα — η ένδειξη ρεύματος γίνεται στατιστικά άχρηστη.

Αυτή η συνάρτηση δειγματοληπτεί **μόνο το ρεύμα, για σταθερά 300 ms** (~4400 δείγματα), και τρέχει αυτόματα όποτε λείπει η τάση. Τυπώνει και το `ptp`, που ξεχωρίζει αμέσως το σήμα από τον θόρυβο:

Λόγος ptp / rms στο pin	Τι σημαίνει
≈ 2.83	Καθαρό ημίτονο — πραγματικό ρεύμα
3 – 4	Πραγματικό ρεύμα με λίγη παραμόρφωση· τυπικό οικιακό φορτίο
10 – 20	Αιχμές, όχι κύμα — **θόρυβος**. Έτσι αποδείχθηκε ότι τα «0.32 A» ήταν ο θόρυβος βάσης.

6. Έξοδος, setup() και loop()

6.1 Διπλή έξοδος: USB CDC και UART0 (γραμμές 94–133)

```
// ------------------------------------------------------------------ ΕΞΟΔΟΣ
// Τα μηνύματα βγαίνουν ταυτόχρονα σε δύο σημεία:
//   * USB CDC       -> ο Serial Monitor μέσω του καλωδίου προγραμματισμού
//   * hardware UART0 -> GPIO21 = TX, GPIO20 = RX, για μετατροπέα USB-TTL
// Έτσι βλέπεις την έξοδο ακόμη κι αν το USB CDC δεν είναι διαθέσιμο (π.χ.
// λάθος ρύθμιση "USB CDC On Boot", ή όταν το ESP32 τροφοδοτείται χωριστά).
#define LOG_TO_UART 1
static const uint32_t LOG_BAUD = 115200;

// Αν το USB CDC είναι απενεργοποιημένο, ο Serial ΕΙΝΑΙ ήδη ο UART0 -> μη
// γράψεις δύο φορές στο ίδιο πράγμα.
#if LOG_TO_UART && ARDUINO_USB_CDC_ON_BOOT
  #define LOG_DUAL 1
#else
  #define LOG_DUAL 0
#endif

static void logBegin() {
  Serial.begin(LOG_BAUD);
#if LOG_DUAL
  Serial0.begin(LOG_BAUD);          // GPIO21 TX / GPIO20 RX
#endif
  // Με USB CDC η θύρα ξαναδηλώνεται στο USB μετά το reset. Χωρίς αυτή την
  // αναμονή, τα πρώτα μηνύματα χάνονται πριν προλάβει να συνδεθεί ο monitor.
  while (!Serial && millis() < 3000) delay(10);
  delay(500);
}

static void logf(const char *fmt, ...) {
  char buf[224];
  va_list ap;
  va_start(ap, fmt);
  vsnprintf(buf, sizeof(buf), fmt, ap);
  va_end(ap);

  Serial.print(buf);
#if LOG_DUAL
  Serial0.print(buf);
#endif
}
```

Κάθε μήνυμα γράφεται στο USB CDC **και** στο hardware UART0 (GPIO21=TX, GPIO20=RX). Ο έλεγχος `#if ARDUINO_USB_CDC_ON_BOOT` είναι απαραίτητος: όταν το «USB CDC On Boot» είναι απενεργοποιημένο, ο `Serial` **είναι ήδη** ο UART0 και κάθε γραμμή θα τυπωνόταν δύο φορές.

Η αναμονή `while (!Serial && millis() < 3000)` υπάρχει επειδή με USB CDC η θύρα ξαναδηλώνεται μετά από κάθε reset. Το όριο των 3 s εμποδίζει το κόλλημα όταν δεν υπάρχει USB.

6.2 setup() (γραμμές 444–470)

```
444 void setup() {
445   logBegin();
446
447   analogReadResolution(12);                          // 0..4095
448   analogSetPinAttenuation(PIN_VOLT, ADC_11db);       // πλήρες εύρος ~0..3.1V
449   analogSetPinAttenuation(PIN_CURR, ADC_11db);
450
451   logf("\n");
452   logf("================================================\n");
453   logf(" Smart Meter – ESP32-C3 | ZMPT101B(GPIO2) SCT-013(GPIO0)\n");
454   logf("================================================\n");
455   logf(" VCAL=%.2f  ICAL=%.2f  PHASECAL=%.2f\n", VCAL, ICAL, PHASECAL);
456 #if LOG_DUAL
457   logf(" Εξοδος    : USB CDC + UART0 (GPIO21=TX, GPIO20=RX) @ %u baud\n", LOG_BAUD);
458 #else
459   logf(" Εξοδος    : UART0 (GPIO21=TX, GPIO20=RX) @ %u baud\n", LOG_BAUD);
460 #endif
461   logf("\n");
462
463 #if USE_MQTT
464   netSetup();
465 #endif
466
467   // Μια "τυφλή" μέτρηση για να σταθεροποιηθούν τα φίλτρα
468   measure();
469   lastEnergy_ms = millis();
470 }
```

ADC_11db είναι η μέγιστη εξασθένηση εισόδου: επιτρέπει σήμα ~0…3.1 V αντί για 0…0.95 V. Στην πράξη ο ADC κορένεται γύρω στα **2853 mV** — αυτό είναι το ταβάνι που ελέγχει το §5.1.

6.3 loop() (γραμμές 473–532)

```
473 void loop() {
474 #if USE_MQTT
475   netLoop();
476 #endif
477
478   Reading r = measure();
479
480   uint32_t now = millis();
481   float dt_h = (now - lastEnergy_ms) / 3600000.0f;   // ms -> ώρες
482   lastEnergy_ms = now;
483
484   if (!r.ok) {
485     logf("[!] Δεν ανιχνεύτηκε σήμα τάσης (timeout). Ελέγξτε ZMPT101B / GPIO2.\n");
486     delay(1000);
487     return;
488   }
489
490   energy_Wh += r.realPower * dt_h;
491
492   // Χωρίς σήμα τάσης η μέτρηση ρεύματος δεν είναι αξιόπιστη (το παράθυρο
493   // ορίζεται από τους μηδενισμούς της τάσης) -> κάνε ξεχωριστή μέτρηση.
494   if (r.vrms <= 0.0f) currentOnlyReport(300);
495
496 #if PRINT_DEBUG
497   // Με φορτίο πάνω από 1 A, τρέξε σάρωση φάσης κάθε 10 δευτερόλεπτα.
498   static uint32_t lastSweep = 0;
499   if (r.irms > 1.0f && millis() - lastSweep > 10000) {
500     lastSweep = millis();
501     phaseSweep();
502   }
503 #endif
504
505 #if PRINT_DEBUG
506   // Crest factor = κορυφή / ενεργός τιμή. Καθαρό ημίτονο -> 1.41.
507   // Πολύ μικρότερο σημαίνει ότι το σήμα "πλακώνει" (saturation του τελεστικού).
508   float vrmsPin_mV = (VCAL > 0) ? (r.vrms / VCAL * 1000.0f) : 0.0f;
509   float crest = (vrmsPin_mV > 1.0f) ? ((r.vmax - r.vmin) / 2.0f) / vrmsPin_mV : 0.0f;
510
511   logf("[dbg] offsetV=%.0fmV offsetI=%.0fmV samples=%d | V: %d..%d mV ptp=%d CF=%.2f %s\n",
512        offsetV, offsetI, r.samples, r.vmin, r.vmax, r.vmax - r.vmin, crest,
513        // Το ταβάνι του ADC είναι ~2850 mV: αν το ακουμπάμε, κόβεται η κορυφή
514        // ΑΝΕΞΑΡΤΗΤΑ από το τι δείχνει το crest factor.
515        (r.vmax > 2800 || r.vmin < 150) ? "<<< ΚΟΡΥΦΗ ΣΤΟ ΤΑΒΑΝΙ - χαμηλωσε κι αλλο"
516          : (crest > 1.33f && crest < 1.50f) ? "OK"
517          : (crest > 0.0f ? "<<< σχημα ακομα στραβο" : ""));
518 #endif
519
520   logf("V=%7.2f V | I=%6.3f A | P=%8.1f W | S=%8.1f VA | PF=%5.2f | E=%9.3f kWh\n",
521                 r.vrms, r.irms, r.realPower, r.apparentPower,
522                 r.powerFactor, energy_Wh / 1000.0);
523
524 #if USE_MQTT
525   if (millis() - lastPublish >= PUBLISH_EVERY_MS) {
526     lastPublish = millis();
527     publishState(r);
528   }
529 #endif
530
531   delay(200);
532 }
```

Το `dt_h` μετριέται με `millis()` και όχι με σταθερά, γιατί ο χρόνος της `measure()` ποικίλλει. Η αφαίρεση σε `uint32_t` δουλεύει σωστά ακόμη και στην υπερχείλιση του millis() (κάθε ~49 μέρες).

Περιορισμός: η `energy_Wh` ζει μόνο στη RAM και μηδενίζεται σε κάθε reset. Αν το Home Assistant κρατάει το ιστορικό δεν πειράζει· αλλιώς αποθήκευσέ την σε NVS/Preferences κάθε λίγα λεπτά.

7. Το κομμάτι του MQTT

Όλος ο κώδικας δικτύου είναι μέσα σε `#if USE_MQTT`.

7.1 Οι ρυθμίσεις (γραμμές 18–36)

```
// ------------------------------------------------------------ WiFi / MQTT
#define USE_MQTT      1      // 0 = μόνο σειριακή, χωρίς δίκτυο
#define HA_DISCOVERY  1      // 1 = αυτόματη αναγνώριση από Home Assistant

#if USE_MQTT
  #include <WiFi.h>
  #include <PubSubClient.h>

  static const char *WIFI_SSID = "TO_DIKTYO_MOU";
  static const char *WIFI_PASS = "TO_PASSWORD_MOU";

  static const char *MQTT_HOST = "192.168.1.17";   // IP του broker
  static const uint16_t MQTT_PORT = 1883;
  static const char *MQTT_USER = "O_XRHSTHS_MOU";        // "" αν δεν χρειάζεται
  static const char *MQTT_PASS = "TO_PASSWORD_TOU_BROKER";

  static const char *MQTT_BASE = "smartmeter";     // ρίζα των topics
  static const uint32_t PUBLISH_EVERY_MS = 5000;   // κάθε πότε δημοσιεύουμε
#endif
```

7.2 Τα μεγέθη που δημοσιεύονται (γραμμές 251–261)

```
struct SensorDef {
  const char *key, *name, *unit, *devClass, *stateClass;
};
static const SensorDef SENSORS[] = {
  {"v",  "Τάση",              "V",   "voltage",        "measurement"},
  {"i",  "Ρεύμα",             "A",   "current",        "measurement"},
  {"p",  "Ισχύς",             "W",   "power",          "measurement"},
  {"s",  "Φαινόμενη ισχύς",   "VA",  "apparent_power", "measurement"},
  {"pf", "Συντελεστής ισχύος", "",   "power_factor",   "measurement"},
  {"e",  "Ενέργεια",          "kWh", "energy",         "total_increasing"},
};
```

Ένας πίνακας αντί για έξι φορές τον ίδιο κώδικα. Το `total_increasing` στην ενέργεια είναι αυτό που την κάνει αποδεκτή από τον **Energy Dashboard**.

7.3 netSetup() (γραμμές 263–279)

```
static void netSetup() {
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);                 // σταθερότερη σύνδεση, λιγότερα stalls
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  uint8_t mac[6];
  WiFi.macAddress(mac);
  snprintf(devId,   sizeof(devId),   "%02x%02x%02x", mac[3], mac[4], mac[5]);
  snprintf(t_state, sizeof(t_state), "%s/%s/state",        MQTT_BASE, devId);
  snprintf(t_avail, sizeof(t_avail), "%s/%s/availability", MQTT_BASE, devId);

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setBufferSize(1024);             // τα payloads του discovery δεν χωράνε στα 256

  logf(" Device ID : %s\n", devId);
  logf(" MQTT topic: %s\n", t_state);
}
```

Η `mqtt.setBufferSize(1024)` είναι **απαραίτητη**: η PubSubClient έχει εξ ορισμού buffer 256 byte και τα payload του discovery είναι ~400. Χωρίς αυτήν οι publish αποτυγχάνουν **σιωπηλά** — καμία ένδειξη λάθους, απλώς τίποτα δεν εμφανίζεται στο HA.

7.4 netLoop(): επανασύνδεση χωρίς να κολλάει ο μετρητής (γραμμές 310–356)

```
310 static void netLoop() {
311   if (WiFi.status() != WL_CONNECTED) {
312     if (millis() - lastNetTry > 10000) {
313       lastNetTry = millis();
314       logf(" [wifi] επανασύνδεση...\n");
315       WiFi.disconnect();
316       WiFi.begin(WIFI_SSID, WIFI_PASS);
317     }
318     return;
319   }
320
321   // Μόλις συνδεθεί το WiFi, τύπωσε μία φορά τη διεύθυνση που πήρε.
322   static bool ipShown = false;
323   if (!ipShown) {
324     ipShown = true;
325     logf(" [wifi] συνδεθηκε: IP=%s  gateway=%s  RSSI=%d dBm\n",
326          WiFi.localIP().toString().c_str(),
327          WiFi.gatewayIP().toString().c_str(),
328          WiFi.RSSI());
329   }
330
331   if (!mqtt.connected()) {
332     if (millis() - lastNetTry > 5000) {
333       lastNetTry = millis();
334       char clientId[32];
335       snprintf(clientId, sizeof(clientId), "%s_%s", MQTT_BASE, devId);
336
337       // Last Will: αν χαθεί η συσκευή, ο broker στέλνει μόνος του "offline".
338       bool ok = mqtt.connect(clientId,
339                              MQTT_USER[0] ? MQTT_USER : nullptr,
340                              MQTT_PASS[0] ? MQTT_PASS : nullptr,
341                              t_avail, 0, true, "offline");
342       if (ok) {
343         logf(" [mqtt] συνδέθηκε στον %s\n", MQTT_HOST);
344         mqtt.publish(t_avail, "online", true);
345 #if HA_DISCOVERY
346         publishDiscovery();
347 #endif
348       } else {
349         logf(" [mqtt] αποτυχία (rc=%d)\n", mqtt.state());
350       }
351     }
352     return;
353   }
354
355   mqtt.loop();
356 }
```

Καμία αναμονή: η συνάρτηση κοιτάζει την ώρα, κάνει **μία** προσπάθεια αν πέρασε αρκετός χρόνος, και επιστρέφει. Ένα `while (!connected) delay(5000)` θα σταματούσε τη μέτρηση κάθε φορά που πέφτει το WiFi.

Last Will. Τα τρία τελευταία ορίσματα της `mqtt.connect()` λένε στον broker να δημοσιεύσει `offline` αν χαθεί η συσκευή, ώστε το HA να τη δείχνει μη διαθέσιμη αντί να κρατάει την τελευταία τιμή.

7.5 publishDiscovery() (γραμμές 284–306)

```
284 static void publishDiscovery() {
285   char topic[96], payload[640], unitField[48];
286
287   for (const SensorDef &s : SENSORS) {
288     snprintf(topic, sizeof(topic),
289              "homeassistant/sensor/%s_%s/%s/config", MQTT_BASE, devId, s.key);
290
291     unitField[0] = '\0';
292     if (s.unit[0]) snprintf(unitField, sizeof(unitField), "\"unit_of_meas\":\"%s\",", s.unit);
293
294     snprintf(payload, sizeof(payload),
295       "{\"name\":\"%s\",\"uniq_id\":\"%s_%s_%s\",\"stat_t\":\"%s\",\"avty_t\":\"%s\","
296       "%s\"dev_cla\":\"%s\",\"stat_cla\":\"%s\",\"val_tpl\":\"{{ value_json.%s }}\","
297       "\"dev\":{\"ids\":[\"%s_%s\"],\"name\":\"Smart Meter\",\"mf\":\"DIY\","
298       "\"mdl\":\"ESP32-C3 + ZMPT101B + SCT-013\"}}",
299       s.name, MQTT_BASE, devId, s.key, t_state, t_avail,
300       unitField, s.devClass, s.stateClass, s.key,
301       MQTT_BASE, devId);
302
303     mqtt.publish(topic, payload, true);   // retained
304   }
305   logf(" [mqtt] στάλθηκε το Home Assistant discovery\n");
306 }
```

Το auto-discovery είναι μια σύμβαση: ένα **retained** JSON σε ειδικό topic και το HA φτιάχνει μόνο του την οντότητα. Το retained είναι το κλειδί — αν το HA ξεκινήσει αργότερα, το βρίσκει εκεί χωρίς να χρειαστεί reset στον μετρητή. Το κοινό `dev.ids` τα ομαδοποιεί σε **μία συσκευή**.

7.6 publishState() (γραμμές 358–368)

```
358 static void publishState(const Reading &r) {
359   if (!mqtt.connected()) return;
360
361   char payload[192];
362   snprintf(payload, sizeof(payload),
363     "{\"v\":%.2f,\"i\":%.3f,\"p\":%.1f,\"s\":%.1f,\"pf\":%.2f,\"e\":%.4f,\"rssi\":%d}",
364     r.vrms, r.irms, r.realPower, r.apparentPower, r.powerFactor,
365     energy_Wh / 1000.0, WiFi.RSSI());
366
367   mqtt.publish(t_state, payload);
368 }
```

Το JSON φτιάχνεται με `snprintf` σε σταθερό buffer, χωρίς ArduinoJson: για έξι αριθμούς δεν αξίζει η επιπλέον βιβλιοθήκη, και η snprintf δεν ξεχειλίζει ποτέ.

8. Σύνοψη βαθμονόμησης

Σταθερά	Τελική τιμή	Επαλήθευση
VCAL	341.0	231.7 V έναντι 233 V πολυμέτρου → 0.55%
ICAL	25.75	τέσσερις συγκρίσεις σε 4.6 και 8.8 A → διασπορά 1%
PHASE_SHIFT	12	PF 0.88 → 1.00· επαληθευμένο σε 1085 W και 2087 W
PHASECAL	1.0	δεν χρειάστηκε λεπτή ρύθμιση
NOISE_I	0.50	μετρημένος θόρυβος βάσης 0.3 A

Το όριο ακρίβειας δεν είναι ο μετρητής, είναι τα όργανα αναφοράς. Δύο συγκρίσεις στο ίδιο φορτίο διέφεραν κατά 1.4% μεταξύ τους. Για καλύτερη βαθμονόμηση χρειάζεται σταθερό γνωστό φορτίο και σύγκριση ισχύος αντί για ρεύμα. Για οικιακή παρακολούθηση το ±2% είναι μια χαρά.

9. Όταν κάτι δεν πάει καλά

Σύμπτωμα	Αιτία και λύση
Ο Serial Monitor μένει κενός	**USB CDC On Boot → Enabled**. Χωρίς αυτό ο Serial βγαίνει στα GPIO20/21. Έλεγξε και ότι η θύρα δεν έχει αλλάξει όνομα.

Σύμπτωμα	Αιτία και λύση
`samples` ~70 αντί για ~2800	Δεν φτάνει κυματομορφή στο GPIO2. Ο βρόχος μετράει τυχαίες εναλλαγές προσήμου του θορύβου.
`offsetV` ≈ 2500 mV	Το ZMPT101B τροφοδοτείται με 5 V. Βάλ' το στα 3.3 V.
`offsetI` πολύ κάτω από 1650	Ο διαιρέτης 10k/10k δεν τροφοδοτείται — έλεγξε την αντίσταση από το 3V3 στο GPIO0.
`max` καρφωμένο στα 2853 mV	Κορεσμός του ADC. Χαμήλωσε το ποτενσιόμετρο *και* βεβαιώσου ότι το κέντρο είναι στα 1650, όχι ψηλότερα.
PF ~0.88 με ωμικό φορτίο	Σφάλμα φάσης. Τρέξε τη σάρωση και πέρασε το PHASE_SHIFT.
Αρνητική ισχύς, kWh που μειώνονται	Το clamp είναι ανάποδα. Γύρνα το 180° ή βάλε `INVERT_CURRENT 1`.
Μαζεύει kWh χωρίς φορτίο	Το `NOISE_I` είναι κάτω από τον θόρυβο βάσης.
`[mqtt] rc=−2`	Δεν βρίσκει τον broker: λάθος IP/θύρα ή δεν συνδέθηκε το WiFi.
`rc=4` ή `rc=5`	Λάθος username/password στον broker.
Το state φτάνει αλλά το HA δεν δείχνει τίποτα	Σχεδόν πάντα ο buffer — `mqtt.setBufferSize(1024)`.